```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('app_versions', function (Blueprint $table) {
            $table->id();
            $table->enum('platform', ['android', 'ios']);
            $table->string('latest_version');
            $table->string('min_supported_version');
            $table->string('store_url')->nullable();
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('app_versions');
    }
};
```

Appversion.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class AppVersion extends Model
```

```
{
    protected $table = 'app_versions';

    protected $fillable = [
        'platform',
        'latest_version',
        'min_supported_version',
        'store_url'
    ];
}
```

Adminappversioncontroller

```
<?php

namespace App\Http\Controllers\Admin;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use App\Models\AppVersion;
use Inertia\Inertia;

class AdminAppVersionController extends Controller
{
    public function index()
    {
        $versions = AppVersion::orderBy('created_at', 'desc')->get();

        return Inertia::render('Admin/AppVersion/Index', [
            'versions' => $versions
        ]);
    }

    public function create()
    {
        // This route is no longer needed as we use modal in Index
        return redirect()->route('admin.app-versions.index');
    }

    public function edit(AppVersion $appVersion)
```

```
    {
        // This route is no longer needed as we use modal in Index
        return redirect()->route('admin.app-versions.index');
    }


    public function store(Request $request)
    {
        $data = $request->validate([
            'platform' => 'required|in:android,ios',
            'latest_version' => 'required|string',
            'min_supported_version' => 'required|string',
            'store_url' => 'nullable|string',
        ]);

        AppVersion::create($data);

        return
redirect()->route('admin.app-versions.index')->with('success', 'App
version added successfully');
    }

    public function update(Request $request, AppVersion $appVersion)
    {
        $data = $request->validate([
            'platform' => 'required|in:android,ios',
            'latest_version' => 'required|string',
            'min_supported_version' => 'required|string',
            'store_url' => 'nullable|string',
        ]);

        $appVersion->update($data);

        return
redirect()->route('admin.app-versions.index')->with('success', 'App
version updated successfully');
    }

    public function destroy(AppVersion $appVersion)
    {
```

```
        $appVersion->delete();
        return
redirect()->route('admin.app-versions.index')->with('success', 'App
version deleted');
    }
}
```

Index.vue

```
<template>
  <AdminLayout>
    <div class="grid gap-5">
      <!-- Page Header -->
      <GlassCard class="fade-up" style="animation-delay:0ms" title="App
Versions Management">
        <div class="flex flex-col gap-4 lg:flex-row lg:items-center
lg:justify-between">
          <div>
            <h2 class="text-xl lg:text-2xl font-bold text-white">App
Versions</h2>
            <p class="text-slate-300 mt-1 text-sm lg:text-base">Manage
application versions and updates</p>
          </div>
          <div class="flex flex-col gap-3 sm:flex-row">
            <button class="btn-primary w-full sm:w-auto"
@click="openCreateModal">
              Create Version
            </button>
          </div>
        </div>
      </GlassCard>

      <!-- App Versions Table -->
      <GlassCard class="fade-up" style="animation-delay:120ms"
title="Versions List">
        <div class="table-container">
          <div class="table">
            <div class="table__head">
              <div class="row">
                <div class="cell">Platform</div>
                <div class="cell">Latest Version</div>
```

```
              <div class="cell">Min Supported</div>
              <div class="cell">Store URL</div>
              <div class="cell">Actions</div>
            </div>
          </div>
          <div class="table__body" v-if="!loading && versions.length">
            <div class="row" v-for="version in versions"
:key="version.id">
              <div class="cell">
                <span class="status-badge"
:class="getPlatformClass(version.platform)">
                  {{ version.platform.toUpperCase() }}
                </span>
              </div>
              <div class="cell">{{ version.latest_version }}</div>
              <div class="cell">{{ version.min_supported_version
}}</div>
              <div class="cell">
                <a v-if="version.store_url" :href="version.store_url"
target="_blank" class="text-blue-400 hover:underline text-sm">
                  View Store
                </a>
                <span v-else class="text-gray-400 text-sm">Not
set</span>
              </div>
              <div class="cell">
                <button @click="editVersion(version)"
class="text-blue-400 hover:text-blue-300 mr-3">Edit</button>
                <button @click="deleteVersion(version.id)"
class="text-red-400 hover:text-red-300">Delete</button>
              </div>
            </div>
          </div>
          <div class="empty" v-else-if="!loading && !versions.length">
            <div class="text-center py-8">
              <p class="text-gray-400 mb-4">No app versions found.
Create your first app version.</p>
              <button @click="openCreateModal"
class="btn-primary">Create First Version</button>
            </div>
```

```
            </div>
            <div class="loading" v-else>Loading...</div>
          </div>
        </div>
      </GlassCard>
    </div>

    <!-- Form Modal -->
    <div v-if="showModal" class="fixed inset-0 z-50 flex items-center
justify-center">
      <div class="absolute inset-0 bg-black/50 backdrop-blur-sm"
@click="closeModal"></div>
      <div class="relative bg-white dark:bg-gray-800 rounded-lg shadow-xl
w-full max-w-md mx-4 max-h-[90vh] overflow-y-auto">
        <div class="sticky top-0 bg-white dark:bg-gray-800 border-b
border-gray-200 dark:border-gray-700 px-6 py-4">
          <div class="flex items-center justify-between">
            <h3 class="text-lg font-semibold text-gray-900
dark:text-white">
              {{ editingVersion ? 'Edit App Version' : 'Create App
Version' }}
            </h3>
            <button @click="closeModal" class="text-gray-400
hover:text-gray-600 dark:hover:text-gray-300">
              <svg class="w-6 h-6" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                <path stroke-linecap="round" stroke-linejoin="round"
stroke-width="2" d="M6 18L18 6M6 6l12 12"></path>
              </svg>
            </button>
          </div>
        </div>

        <form @submit.prevent="submit" class="p-6 space-y-4">
          <div>
            <label class="block text-sm font-medium text-gray-700
dark:text-gray-300 mb-2">Platform</label>
            <select v-model="form.platform" class="w-full px-3 py-2 border
border-gray-300 dark:border-gray-600 rounded-lg focus:ring-2
```

```
focus:ring-blue-500 focus:border-blue-500 dark:bg-gray-700
dark:text-white">
              <option value="android">Android</option>
              <option value="ios">iOS</option>
            </select>
          </div>

          <div>
            <label class="block text-sm font-medium text-gray-700
dark:text-gray-300 mb-2">Latest Version</label>
            <input type="text" v-model="form.latest_version" class="w-full
px-3 py-2 border border-gray-300 dark:border-gray-600 rounded-lg
focus:ring-2 focus:ring-blue-500 focus:border-blue-500 dark:bg-gray-700
dark:text-white" placeholder="e.g., 1.0.0" required />
          </div>

          <div>
            <label class="block text-sm font-medium text-gray-700
dark:text-gray-300 mb-2">Minimum Supported Version</label>
            <input type="text" v-model="form.min_supported_version"
class="w-full px-3 py-2 border border-gray-300 dark:border-gray-600
rounded-lg focus:ring-2 focus:ring-blue-500 focus:border-blue-500
dark:bg-gray-700 dark:text-white" placeholder="e.g., 1.0.0" required />
          </div>

          <div>
            <label class="block text-sm font-medium text-gray-700
dark:text-gray-300 mb-2">Store URL (Optional)</label>
            <input type="url" v-model="form.store_url" class="w-full px-3
py-2 border border-gray-300 dark:border-gray-600 rounded-lg focus:ring-2
focus:ring-blue-500 focus:border-blue-500 dark:bg-gray-700
dark:text-white"
placeholder="https://play.google.com/store/apps/details?id=com.app" />
          </div>

          <div class="flex justify-end space-x-3 pt-4">
            <button type="button" @click="closeModal" class="px-4 py-2
text-gray-700 dark:text-gray-300 bg-gray-200 dark:bg-gray-600 rounded-lg
hover:bg-gray-300 dark:hover:bg-gray-500 transition-colors">
              Cancel
```

```
            </button>
            <button type="submit" class="px-4 py-2 bg-blue-600 text-white
rounded-lg hover:bg-blue-700 transition-colors">
              {{ editingVersion ? 'Update' : 'Create' }}
            </button>
          </div>
        </form>
      </div>
    </div>
  </AdminLayout>
</template>

<script setup>
import AdminLayout from '@/Layouts/AdminLayout.vue';
import GlassCard from '@/Components/GlassCard.vue';
import { ref, onMounted } from 'vue';
import { router } from '@inertiajs/vue3';

const props = defineProps({
  versions: { type: Array, default: () => [] }
});

const loading = ref(false);
const showModal = ref(false);
const editingVersion = ref(null);

const form = ref({
  platform: 'android',
  latest_version: '',
  min_supported_version: '',
  store_url: '',
});

function getPlatformClass(platform) {
  return platform === 'android' ? 'platform-android' : 'platform-ios';
}

function openCreateModal() {
  editingVersion.value = null;
  form.value = {
```

```
    platform: 'android',
    latest_version: '',
    min_supported_version: '',
    store_url: '',
  };
  showModal.value = true;
}

function editVersion(version) {
  editingVersion.value = version;
  form.value = {
    platform: version.platform,
    latest_version: version.latest_version,
    min_supported_version: version.min_supported_version,
    store_url: version.store_url || '',
  };
  showModal.value = true;
}

function closeModal() {
  showModal.value = false;
  editingVersion.value = null;
}

function submit() {
  if (editingVersion.value) {
    router.put(`/admin/app-versions/${editingVersion.value.id}`,
form.value, {
      onSuccess: () => {
        closeModal();
      }
    });
  } else {
    router.post('/admin/app-versions/store', form.value, {
      onSuccess: () => {
        closeModal();
      }
    });
  }
}
```

```
function deleteVersion(id) {
  if (confirm('Are you sure you want to delete this app version?')) {
    router.delete(`/admin/app-versions/${id}`, {
      onSuccess: () => {
        // Page will automatically reload with updated data
      }
    });
  }
}
</script>

<style scoped>
.table {
  width: 100%;
  border-collapse: collapse;
}

.table__head {
  background-color: rgba(255, 255, 255, 0.05);
}

.table__head .row {
  display: grid;
  grid-template-columns: 1fr 1.2fr 1.2fr 1.5fr 1fr;
  gap: 1rem;
  padding: 1rem;
  border-bottom: 1px solid rgba(255, 255, 255, 0.1);
}

.table__body .row {
  display: grid;
  grid-template-columns: 1fr 1.2fr 1.2fr 1.5fr 1fr;
  gap: 1rem;
  padding: 1rem;
  border-bottom: 1px solid rgba(255, 255, 255, 0.05);
  transition: background-color 0.2s;
}

.table__body .row:hover {
```

```
  background-color: rgba(255, 255, 255, 0.02);
}

.cell {
  display: flex;
  align-items: center;
  color: #e2e8f0;
}

.empty {
  text-align: center;
  padding: 3rem;
  color: #94a3b8;
}

.loading {
  text-align: center;
  padding: 3rem;
  color: #94a3b8;
}

.btn-primary {
  background-color: #3b82f6;
  color: white;
  padding: 0.5rem 1rem;
  border-radius: 0.375rem;
  font-weight: 500;
  transition: background-color 0.2s;
}

.btn-primary:hover {
  background-color: #2563eb;
}

.status-badge {
  display: inline-block;
  padding: 0.25rem 0.5rem;
  border-radius: 0.25rem;
  font-size: 0.75rem;
  font-weight: 600;
```

```
  text-transform: uppercase;
}

.platform-android {
  background-color: #3ddc84;
  color: #000;
}

.platform-ios {
  background-color: #007aff;
  color: #fff;
}

/* Responsive Design */
@media (max-width: 1024px) {
  .table__head .row,
  .table__body .row {
    grid-template-columns: 1fr 1.2fr 1.2fr 1fr;
  }
  .row .cell:nth-child(5) {
    display: none;
  }
}

@media (max-width: 768px) {
  .table__head .row,
  .table__body .row {
    grid-template-columns: 1fr 1.5fr 1fr;
  }
  .row .cell:nth-child(4),
  .row .cell:nth-child(5) {
    display: none;
  }
}

@media (max-width: 640px) {
  .table__head .row,
  .table__body .row {
    grid-template-columns: 1fr 1fr;
    gap: 4px;
```

```
    padding: 8px 4px;
  }
  .row .cell:nth-child(3),
  .row .cell:nth-child(4),
  .row .cell:nth-child(5) {
    display: none;
  }
}

@keyframes fadeUp {
  0% { opacity: 0; transform: translateY(18px) scale(0.98); }
  100% { opacity: 1; transform: translateY(0) scale(1); }
}
.fade-up { animation: fadeUp 700ms cubic-bezier(.21,.61,.35,1) both; }
</style>
```

Api

```
use App\Http\Controllers\Api\AppVersionController;
```

```
Route::post('app-version/check', [AppVersionController::class,
'checkVersion']);
```

```
// App Version Management (Protected routes)
        Route::prefix('app-version')->group(function () {
            Route::get('/', [AppVersionController::class, 'index']);
            Route::post('/', [AppVersionController::class, 'store']);
            Route::put('/{id}', [AppVersionController::class, 'update']);
            Route::delete('/{id}', [AppVersionController::class,
'destroy']);
            Route::get('/platform/{platform}',
[AppVersionController::class, 'getByPlatform']);
        });
```

Web

```
use App\Http\Controllers\Admin\AdminAppVersionController;
```

```
Route::prefix('app-versions')->group(function () {
            Route::get('/', [AdminAppVersionController::class,
'index'])->name('admin.app-versions.index');
            Route::get('/create', [AdminAppVersionController::class,
'create'])->name('admin.app-versions.create');
```

```
            Route::post('/store', [AdminAppVersionController::class,
'store'])->name('admin.app-versions.store');
            Route::get('/{appVersion}/edit',
[AdminAppVersionController::class,
'edit'])->name('admin.app-versions.edit');
            Route::put('/{appVersion}', [AdminAppVersionController::class,
'update'])->name('admin.app-versions.update');
            Route::delete('/{appVersion}',
[AdminAppVersionController::class,
'destroy'])->name('admin.app-versions.destroy');
        });
```

Appversionrepositoryinterface

```
<?php

namespace App\Repositories\Contracts;

use App\Models\AppVersion;

interface AppVersionRepositoryInterface
{
    public function getAll();
    public function getById(int $id);
    public function getByPlatform(string $platform);
    public function create(array $data);
    public function update(int $id, array $data);
    public function delete(int $id);
    public function getLatestByPlatform(string $platform);
}
```

Appversionrepository

```
<?php

namespace App\Repositories\Eloquent;

use App\Models\AppVersion;
use App\Repositories\Contracts\AppVersionRepositoryInterface;
```

```php
class AppVersionRepository implements AppVersionRepositoryInterface
{
    protected $model;

    public function __construct(AppVersion $model)
    {
        $this->model = $model;
    }

    public function getAll()
    {
        return $this->model->orderBy('created_at', 'desc')->get();
    }

    public function getById(int $id)
    {
        return $this->model->find($id);
    }

    public function getByPlatform(string $platform)
    {
        return $this->model->where('platform', $platform)->get();
    }

    public function create(array $data)
    {
        return $this->model->create($data);
    }

    public function update(int $id, array $data)
    {
        $appVersion = $this->getById($id);
        if (!$appVersion) {
            return null;
        }
        $appVersion->update($data);
        return $appVersion;
    }
```

```
    public function delete(int $id)
    {
        $appVersion = $this->getById($id);
        if (!$appVersion) {
            return false;
        }
        return $appVersion->delete();
    }

    public function getLatestByPlatform(string $platform)
    {
        return $this->model->where('platform', $platform)
            ->orderBy('created_at', 'desc')
            ->first();
    }
}
```

Appversionserice

```
<?php

namespace App\Services;

use App\Repositories\Contracts\AppVersionRepositoryInterface;
use Illuminate\Http\Request;
use Exception;

class AppVersionService
{
    protected $repository;

    public function __construct(AppVersionRepositoryInterface $repository)
    {
        $this->repository = $repository;
    }

    public function getAllVersions()
    {
        try {
```

```
            return $this->repository->getAll();
        } catch (Exception $e) {
            throw new Exception('Failed to retrieve app versions: ' .
$e->getMessage());
        }
    }

    public function getVersionById(int $id)
    {
        try {
            $version = $this->repository->getById($id);
            if (!$version) {
                throw new Exception('App version not found');
            }
            return $version;
        } catch (Exception $e) {
            throw new Exception('Failed to retrieve app version: ' .
$e->getMessage());
        }
    }

    public function getVersionByPlatform(string $platform)
    {
        try {
            if (!in_array($platform, ['android', 'ios'])) {
                throw new Exception('Invalid platform specified');
            }
            return $this->repository->getByPlatform($platform);
        } catch (Exception $e) {
            throw new Exception('Failed to retrieve versions for platform:
' . $e->getMessage());
        }
    }

    public function createVersion(array $data)
    {
        try {
            // Validate platform
            if (!in_array($data['platform'], ['android', 'ios'])) {
                throw new Exception('Invalid platform specified');
```

```
            }

            // Validate version format (basic semantic versioning)
            if (!$this->isValidVersion($data['latest_version'])) {
                throw new Exception('Invalid latest version format');
            }

            if (!$this->isValidVersion($data['min_supported_version'])) {
                throw new Exception('Invalid minimum supported version
format');
            }

            return $this->repository->create($data);
        } catch (Exception $e) {
            throw new Exception('Failed to create app version: ' .
$e->getMessage());
        }
    }

    public function updateVersion(int $id, array $data)
    {
        try {
            // Validate platform if provided
            if (isset($data['platform']) && !in_array($data['platform'],
['android', 'ios'])) {
                throw new Exception('Invalid platform specified');
            }

            // Validate version format if provided
            if (isset($data['latest_version']) &&
!$this->isValidVersion($data['latest_version'])) {
                throw new Exception('Invalid latest version format');
            }

            if (isset($data['min_supported_version']) &&
!$this->isValidVersion($data['min_supported_version'])) {
                throw new Exception('Invalid minimum supported version
format');
            }
```

```
            $version = $this->repository->update($id, $data);
            if (!$version) {
                throw new Exception('App version not found');
            }
            return $version;
        } catch (Exception $e) {
            throw new Exception('Failed to update app version: ' .
$e->getMessage());
        }
    }

    public function deleteVersion(int $id)
    {
        try {
            $result = $this->repository->delete($id);
            if (!$result) {
                throw new Exception('App version not found');
            }
            return true;
        } catch (Exception $e) {
            throw new Exception('Failed to delete app version: ' .
$e->getMessage());
        }
    }

    public function checkVersion(string $platform, string $currentVersion)
    {
        try {
            if (!in_array($platform, ['android', 'ios'])) {
                throw new Exception('Invalid platform specified');
            }

            if (!$this->isValidVersion($currentVersion)) {
                throw new Exception('Invalid current version format');
            }

            $latestVersion =
$this->repository->getLatestByPlatform($platform);

            if (!$latestVersion) {
```

```
                return [
                    'status' => 'no_version_found',
                    'message' => 'No version information available for
this platform',
                    'update_required' => false,
                    'force_update' => false,
                    'store_url' => null
                ];
            }

            $comparison = $this->compareVersions($currentVersion,
$latestVersion->latest_version);
            $minSupportedComparison =
$this->compareVersions($currentVersion,
$latestVersion->min_supported_version);

            return [
                'status' => 'success',
                'current_version' => $currentVersion,
                'latest_version' => $latestVersion->latest_version,
                'min_supported_version' =>
$latestVersion->min_supported_version,
                'update_required' => $comparison < 0,
                'force_update' => $minSupportedComparison < 0,
                'store_url' => $latestVersion->store_url,
                'message' => $this->getVersionMessage($comparison,
$minSupportedComparison)
            ];
        } catch (Exception $e) {
            throw new Exception('Failed to check version: ' .
$e->getMessage());
        }
    }

    private function isValidVersion(string $version)
    {
        return preg_match('/^\d+\.\d+\.\d+$/', $version);
    }

    private function compareVersions(string $version1, string $version2)
```

```
    {
        $v1Parts = explode('.', $version1);
        $v2Parts = explode('.', $version2);

        for ($i = 0; $i < 3; $i++) {
            $v1Part = (int)($v1Parts[$i] ?? 0);
            $v2Part = (int)($v2Parts[$i] ?? 0);

            if ($v1Part < $v2Part) {
                return -1;
            } elseif ($v1Part > $v2Part) {
                return 1;
            }
        }

        return 0;
    }

    private function getVersionMessage(int $comparison, int
$minSupportedComparison)
    {
        if ($minSupportedComparison < 0) {
            return 'This version is no longer supported. Please update
immediately.';
        } elseif ($comparison < 0) {
            return 'A new version is available. Please update for the best
experience.';
        } else {
            return 'You are using the latest version.';
        }
    }
}
```

Appversioncontroller

```
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
```

```php
use App\Http\Requests\CheckVersionRequest;
use App\Services\AppVersionService;
use Illuminate\Http\Request;
use Illuminate\Http\JsonResponse;
use Exception;

class AppVersionController extends Controller
{
    protected $service;

    public function __construct(AppVersionService $service)
    {
        $this->service = $service;
    }

    /**
     * Get all app versions
     */
    public function index(): JsonResponse
    {
        try {
            $versions = $this->service->getAllVersions();

            return response()->json([
                'status' => 'success',
                'data' => $versions,
                'message' => 'App versions retrieved successfully'
            ], 200);
        } catch (Exception $e) {
            return response()->json([
                'status' => 'error',
                'message' => $e->getMessage()
            ], 500);
        }
    }

    /**
     * Create a new app version
     */
    public function store(Request $request): JsonResponse
```

```
    {
        try {
            $data = $request->validate([
                'platform' => 'required|in:android,ios',
                'latest_version' => 'required|string',
                'min_supported_version' => 'required|string',
                'store_url' => 'nullable|string'
            ]);

            $version = $this->service->createVersion($data);

            return response()->json([
                'status' => 'success',
                'data' => $version,
                'message' => 'App version created successfully'
            ], 201);
        } catch (Exception $e) {
            return response()->json([
                'status' => 'error',
                'message' => $e->getMessage()
            ], 400);
        }
    }

    /**
     * Update an app version
     */
    public function update(Request $request, int $id): JsonResponse
    {
        try {
            $data = $request->validate([
                'platform' => 'sometimes|required|in:android,ios',
                'latest_version' => 'sometimes|required|string',
                'min_supported_version' => 'sometimes|required|string',
                'store_url' => 'sometimes|nullable|string'
            ]);

            $version = $this->service->updateVersion($id, $data);

            return response()->json([
```

```
                'status' => 'success',
                'data' => $version,
                'message' => 'App version updated successfully'
            ], 200);
        } catch (Exception $e) {
            return response()->json([
                'status' => 'error',
                'message' => $e->getMessage()
            ], 400);
        }
    }

    /**
     * Check app version compatibility
     */
    public function checkVersion(Request $request): JsonResponse
    {
        try {
            $data = $request->validate([
                'platform' => 'required|in:android,ios',
                'current_version' => 'required|string'
            ]);

            // Get version from header if not in request body
            $currentVersion = $data['current_version'];
            if ($request->hasHeader('App-Version')) {
                $currentVersion = $request->header('App-Version');
            }

            $result = $this->service->checkVersion($data['platform'],
$currentVersion);

            // Return 426 if update is required
            if ($result['update_required']) {
                return response()->json([
                    'status' => 'update_required',
                    'data' => $result,
                    'message' => $result['message']
                ], 426);
            }
```

```
            return response()->json([
                'status' => $result['status'],
                'data' => $result,
                'message' => $result['message']
            ], 200);
        } catch (Exception $e) {
            return response()->json([
                'status' => 'error',
                'message' => $e->getMessage()
            ], 400);
        }
    }

    /**
     * Get versions by platform
     */
    public function getByPlatform(string $platform): JsonResponse
    {
        try {
            $versions = $this->service->getVersionByPlatform($platform);

            return response()->json([
                'status' => 'success',
                'data' => $versions,
                'message' => "Versions for {$platform} retrieved
successfully"
            ], 200);
        } catch (Exception $e) {
            return response()->json([
                'status' => 'error',
                'message' => $e->getMessage()
            ], 400);
        }
    }

    /**
     * Delete an app version
     */
    public function destroy(int $id): JsonResponse
```

```
    {
        try {
            $this->service->deleteVersion($id);

            return response()->json([
                'status' => 'success',
                'message' => 'App version deleted successfully'
            ], 200);
        } catch (Exception $e) {
            return response()->json([
                'status' => 'error',
                'message' => $e->getMessage()
            ], 400);
        }
    }
}
```

Checkappversion.php

```
<?php

namespace App\Http\Middleware;

use App\Services\AppVersionService;
use Closure;
use Illuminate\Http\Request;
use Illuminate\Http\JsonResponse;

class CheckAppVersion
{
    protected $service;

    public function __construct(AppVersionService $service)
    {
        $this->service = $service;
    }

    /**
     * Handle an incoming request.
     */
```

```php
    public function handle(Request $request, Closure $next)
    {
        // Skip version check for version endpoint itself and auth
endpoints
        $excludedPaths = [
            'api/v1/app-version/check',
            'api/v1/auth',
            'api/v1/sports-image',
            'api/v1/continue-guest'
        ];

        foreach ($excludedPaths as $path) {
            if (str_contains($request->path(), $path)) {
                return $next($request);
            }
        }

        // Get app version from header
        $appVersion = $request->header('App-Version');
        $platform = $request->header('App-Platform', 'android'); //
Default to android

        if (!$appVersion) {
            return response()->json([
                'status' => 'error',
                'message' => 'App-Version header is required'
            ], 400);
        }

        try {
            $result = $this->service->checkVersion($platform,
$appVersion);

            // Add version info to response headers
            $response = $next($request);

            if ($response instanceof JsonResponse) {
                $response->headers->set('X-App-Version-Latest',
$result['latest_version'] ?? 'unknown');
```

```
                $response->headers->set('X-App-Version-Min-Supported',
$result['min_supported_version'] ?? 'unknown');
                $response->headers->set('X-App-Update-Required',
$result['update_required'] ? 'true' : 'false');
                $response->headers->set('X-App-Force-Update',
($result['force_update'] ?? false) ? 'true' : 'false');
                $response->headers->set('X-App-Store-URL',
$result['store_url'] ?? '');
            }

            // Return 426 if update is required
            if ($result['update_required']) {
                return response()->json([
                    'status' => 'update_required',
                    'data' => $result,
                    'message' => $result['message']
                ], 426);
            }

            return $response;
        } catch (\Exception $e) {
            // Log error but don't block the request
            \Log::warning('Version check failed: ' . $e->getMessage());
            return $next($request);
        }
    }
}
```

Providers.php

```
<?php

return [
    App\Providers\AppServiceProvider::class,
    App\Providers\RepositoryServiceProvider::class,
];
```

Checkversionrequest.php

```
<?php
```

```php
namespace App\Http\Requests\Api;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Contracts\Validation\Validator;
use Illuminate\Http\Exceptions\HttpResponseException;

class CheckVersionRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return true;
    }

    /**
     * Get the validation rules that apply to the request.
     */
    public function rules(): array
    {
        return [
            'platform' => 'required|in:android,ios',
            'current_version' =>
'sometimes|required|string|regex:/^\d+\.\d+\.\d+$/',
        ];
    }

    /**
     * Get custom error messages for validation rules.
     */
    public function messages(): array
    {
        return [
            'platform.required' => 'Platform is required',
            'platform.in' => 'Platform must be either android or ios',
            'current_version.required' => 'Current version is required',
            'current_version.regex' => 'Version must be in format x.x.x
(e.g., 1.0.0)',
```

```
        ];
    }

    /**
     * Handle a failed validation attempt.
     */
    protected function failedValidation(Validator $validator)
    {
        throw new HttpResponseException(
            response()->json([
                'status' => 'error',
                'message' => 'Validation failed',
                'errors' => $validator->errors()
            ], 422)
        );
    }
}
```

Appversionresource

```
<?php

namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class AppVersionResource extends JsonResource
{
    /**
     * Transform the resource into an array.
     */
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'platform' => $this->platform,
            'latest_version' => $this->latest_version,
            'min_supported_version' => $this->min_supported_version,
            'store_url' => $this->store_url,
```

```
            'created_at' => $this->created_at,
            'updated_at' => $this->updated_at,
        ];
    }
}
```

App.php

```
        $middleware->api(append: [
            \App\Http\Middleware\CheckAppVersion::class,
        ]);
```